

Pipeline CI/CD com GitHub Actions

Material instrucional - projeto full-stack FeedbackHub CI

Objetivo do material

Construir uma aplicação full-stack simples, configurar validações locais e criar um pipeline de CI/CD no GitHub Actions para validar backend, frontend, documentação e deploy simulado.

Ao final da prática, você deverá ser capaz de explicar o papel de cada etapa do pipeline, interpretar falhas no GitHub Actions e corrigir problemas comuns antes de realizar o merge de uma alteração.

Resumo da prática

Bloco	O que será feito	Resultado esperado
1	Criar a estrutura full-stack do projeto	Repositório feedbackhub-ci organizado em backend, frontend, docs e workflow
2	Configurar backend Node.js + Express	API com endpoints de feedbacks, testes, lint e build
3	Configurar frontend React + Vite	Interface demonstrativa com teste, lint e build
4	Criar documentação técnica mínima	README, INSTALLATION, ARCHITECTURE, Mermaid e OpenAPI
5	Criar pipeline GitHub Actions	Jobs separados para backend, frontend, documentação e deploy simulado
6	Simular falhas controladas	Interpretação de erros e correção do pipeline

1. Contexto da atividade

Nesta prática, será criada uma aplicação demonstrativa chamada FeedbackHub CI. O projeto simula uma aplicação full-stack para coleta e gestão de feedbacks de usuários.

O foco principal não é desenvolver uma aplicação de produção, mas compreender como um pipeline automatizado pode validar código, testes, build e documentação antes que uma alteração seja integrada à branch principal.

Stack utilizada

Camada	Tecnologias	Finalidade
Backend	Node.js + Express	Expor endpoints REST e regras básicas da API
Frontend	React + Vite	Renderizar a interface demonstrativa
Testes backend	Jest + Supertest	Validar endpoints e respostas da API
Testes frontend	Vitest + React Testing Library	Validar renderização de elementos da interface
Lint	ESLint	Detectar problemas de estilo e qualidade de código
Pipeline	GitHub Actions	Executar validações automaticamente no GitHub
Documentação	README, OpenAPI e Mermaid	Registrar uso, arquitetura, contrato da API e fluxo

2. Conceitos essenciais antes da prática

2.1 O que é CI?

CI significa Continuous Integration, ou Integração Contínua. Na prática, é o processo de validar automaticamente cada alteração enviada ao repositório, principalmente por meio de lint, testes e build.

A CI reduz o risco de integrar código quebrado, porque problemas são detectados antes do merge.

2.2 O que é CD?

CD pode significar Continuous Delivery ou Continuous Deployment. Nesta prática, será usado um deploy simulado, ou seja, o pipeline terá uma etapa final que representa a entrega, mas sem publicar a aplicação em um servidor real.

2.3 O que é GitHub Actions?

GitHub Actions é o mecanismo do GitHub para criar automações baseadas em eventos do repositório. Um workflow pode rodar quando ocorre push, pull request, criação de tag ou execução manual.

2.4 Principais termos do GitHub Actions

Termo	Significado
Workflow	Arquivo YAML que define a automação. Fica em <code>.github/workflows/</code> .
Evento	Condição que dispara o workflow, como push ou pull_request.
Job	Grupo de etapas executado em um ambiente, como ubuntu-latest.
Step	Uma ação individual dentro de um job, como instalar dependências ou rodar testes.
Runner	Máquina temporária usada para executar o workflow.

needs	Dependência entre jobs. Permite executar um job somente depois de outros.
-------	---

Ideia central

O pipeline não substitui a revisão técnica. Ele automatiza verificações repetitivas e ajuda a detectar problemas cedo, antes que cheguem à branch main.

3. Estrutura do projeto

A aplicação será organizada em uma estrutura full-stack. Cada área do projeto terá uma responsabilidade clara.

```

feedbackhub-ci/
├── .github/workflows/ci.yml
├── backend/
│   ├── src/
│   │   ├── app.js
│   │   └── server.js
│   ├── tests/
│   │   └── feedbacks.test.js
│   ├── package.json
│   └── openapi.yaml
├── frontend/
│   ├── src/
│   │   ├── App.jsx
│   │   ├── main.jsx
│   │   ├── pages/
│   │   └── components/
│   ├── index.html
│   ├── package.json
│   └── vite.config.js
├── docs/
│   ├── ARCHITECTURE.md
│   ├── INSTALLATION.md
│   └── diagrams/api-flow.md
├── README.md
└── .gitignore

```

Descrição das principais pastas

Pasta ou arquivo	Responsabilidade
backend/	Código da API, testes de endpoints e contrato OpenAPI.
frontend/	Código da interface React, testes de renderização e build.
docs/	Documentação técnica complementar.
.github/workflows/ci.yml	Definição do pipeline CI/CD.
README.md	Ponto de entrada do projeto.

4. Criação do repositório e estrutura local

Crie um repositório vazio no GitHub chamado feedbackhub-ci. Não marque README, .gitignore ou licença, pois os arquivos serão criados manualmente durante a prática.

4.1 Criar pasta local

```

cd ~/Documentos/CURSOS/AI_Devs_T2
mkdir feedbackhub-ci
cd feedbackhub-ci
pwd

```

O comando `pwd` deve mostrar um caminho que termina em `feedbackhub-ci`.

4.2 Criar estrutura de pastas e arquivos

```
mkdir -p .github/workflows
mkdir -p backend/src backend/tests
mkdir -p frontend/src/pages frontend/src/components
mkdir -p docs/diagrams

touch .github/workflows/ci.yml
touch backend/src/app.js backend/src/server.js
touch backend/tests/feedbacks.test.js backend/openapi.yaml
touch frontend/src/App.jsx frontend/src/main.jsx
touch frontend/src/pages/Dashboard.jsx frontend/src/pages/FeedbackList.jsx
touch frontend/src/pages/NewFeedback.jsx
touch frontend/src/components/FeedbackCard.jsx
touch frontend/index.html frontend/vite.config.js
touch docs/ARCHITECTURE.md docs/INSTALLATION.md docs/diagrams/api-flow.md
touch README.md .gitignore
```

5. Backend: API Node.js + Express

O backend será uma API REST simples para gerenciar feedbacks. Os dados ficarão em memória, porque o objetivo da prática é validar pipeline, testes e documentação, não persistência em banco de dados.

5.1 Criar o backend

```
cd backend
npm init -y
npm install express cors
npm install --save-dev jest supertest eslint @eslint/js globals
```

5.2 Scripts esperados do backend

O arquivo `backend/package.json` deve conter scripts que o pipeline conseguirá executar de forma automatizada.

```
{
  "scripts": {
    "start": "node src/server.js",
    "test": "jest",
    "build": "node -c src/app.js && node -c src/server.js",
    "lint": "eslint src tests"
  },
  "jest": {
    "testEnvironment": "node"
  }
}
```

Script	O que valida
<code>npm run lint</code>	Qualidade e padrões básicos de código com ESLint.
<code>npm test</code>	Comportamento dos endpoints usando Jest e Supertest.
<code>npm run build</code>	Validação sintática dos arquivos principais do backend.
<code>npm start</code>	Inicialização local da API.

5.3 Endpoints esperados da API

Método	Endpoint	Objetivo
GET	<code>/health</code>	Verificar se a API está funcionando.
GET	<code>/feedbacks</code>	Listar feedbacks, com filtros opcionais por status e category.
POST	<code>/feedbacks</code>	Cadastrar novo feedback.
GET	<code>/feedbacks/{id}</code>	Consultar feedback por ID.

PATCH	/feedbacks/{id}/status	Atualizar status de um feedback.
PATCH	/feedbacks/{id}/archive	Arquivar um feedback.
POST	/feedbacks/{id}/replies	Cadastrar resposta para um feedback.
GET	/feedbacks/{id}/replies	Listar respostas de um feedback.
GET	/metrics	Consultar métricas gerais da aplicação.

Atenção

O endpoint `/feedbacks/{id}/archive` será usado como validação explícita no pipeline. Se ele não estiver documentado no `openapi.yaml`, o job de backend deve falhar.

6. Testes do backend com Jest e Supertest

Os testes do backend validam se a API responde corretamente. O Supertest permite chamar a aplicação Express sem precisar subir um servidor manualmente.

6.1 Cenários mínimos que devem existir

- GET `/health` deve retornar 200 e status ok.
- GET `/feedbacks` deve retornar uma lista.
- GET `/feedbacks?status=new` deve retornar apenas feedbacks com status new.
- GET `/feedbacks?status=invalid` deve retornar 400.
- POST `/feedbacks` deve criar um feedback válido.
- POST `/feedbacks` sem `userName` deve retornar 400.
- GET `/feedbacks/999` deve retornar 404.
- PATCH `/feedbacks/1/archive` deve alterar o status para archived.
- GET `/metrics` deve retornar métricas gerais.

6.2 Exemplo de teste com Supertest

```
const request = require('supertest');
const app = require('../src/app');

test('deve retornar status de saúde da API', async () => {
  const response = await request(app).get('/health');

  expect(response.status).toBe(200);
  expect(response.body).toHaveProperty('status', 'ok');
  expect(response.body).toHaveProperty('service', 'feedbackhub-api');
});
```

Esse teste segue uma estrutura simples: executa uma requisição, recebe a resposta e valida status code e propriedades do JSON.

7. OpenAPI: contrato técnico da API

O arquivo `backend/openapi.yaml` documenta o contrato da API. Ele informa quais endpoints existem, quais parâmetros são aceitos, quais bodies são esperados e quais respostas podem ser retornadas.

7.1 Versão mínima inicial

Não deixe o arquivo vazio. Uma versão mínima ajuda a IA a manter o formato correto ao completar a documentação.

```
openapi: 3.0.0
```

```
info:
  title: FeedbackHub API
  version: 1.0.0
  description: API para coleta e gestão de feedbacks de usuários.
paths: {}
```

7.2 Prompt para atualizar o OpenAPI com IA

Analise o backend em backend/src/app.js.

Atualize o arquivo backend/openapi.yaml documentando obrigatoriamente todos os endpoints existentes da API FeedbackHub CI:

- GET /health
- GET /feedbacks
- POST /feedbacks
- GET /feedbacks/{id}
- PATCH /feedbacks/{id}/status
- PATCH /feedbacks/{id}/archive
- POST /feedbacks/{id}/replies
- GET /feedbacks/{id}/replies
- GET /metrics

Para cada endpoint, inclua summary, descrição resumida, parâmetros de path ou query, requestBody quando existir, status codes esperados e descrição das respostas.

Inclua schemas reutilizáveis em components/schemas para Feedback, CreateFeedbackRequest, UpdateFeedbackStatusRequest, Reply, CreateReplyRequest, Metrics e ErrorResponse.

Regras:

- Use OpenAPI 3.0.0.
- Não remova /feedbacks/{id}/archive.
- Não invente endpoints que não existem em backend/src/app.js.
- O arquivo deve ser compatível com validações por grep no GitHub Actions.
- Gere apenas o conteúdo final do arquivo backend/openapi.yaml.

8. Frontend: React + Vite

O frontend será uma interface simples para representar o produto full-stack. Ele não precisa consumir a API de forma real nesta prática; o objetivo é ter uma camada frontend com lint, testes e build.

8.1 Criar o frontend

```
cd ../frontend
npm init -y
npm install @vitejs/plugin-react vite react react-dom
npm install --save-dev eslint vitest @testing-library/react @testing-library/jest-dom jsdom eslint-plugin-react
```

8.2 Scripts esperados do frontend

```
{
  "type": "module",
  "scripts": {
    "dev": "vite --host 0.0.0.0",
    "test": "vitest run",
    "build": "vite build",
    "lint": "eslint src --ext .js, .jsx"
  }
}
```

8.3 Telas e componentes esperados

Arquivo	Responsabilidade
src/App.jsx	Compor a página principal da aplicação.

src/main.jsx	Renderizar a aplicação React.
src/pages/Dashboard.jsx	Exibir métricas gerais.
src/pages/FeedbackList.jsx	Listar feedbacks.
src/pages/NewFeedback.jsx	Representar formulário de novo feedback.
src/components/FeedbackCard.jsx	Representar um feedback individual.

8.4 Teste mínimo do frontend

O teste do frontend valida se elementos importantes aparecem na tela. Isso será útil para simular uma falha quando o título principal for alterado.

```
import { render, screen } from '@testing-library/react';
import '@testing-library/jest-dom';
import App from './App.jsx';

test('deve renderizar o título principal da aplicação', () => {
  render(<App />);
  expect(screen.getByText('Gestão de feedbacks de usuários')).toBeInTheDocument();
});

test('deve renderizar a seção de novo feedback', () => {
  render(<App />);
  expect(screen.getByText('Novo feedback')).toBeInTheDocument();
});
```

9. Documentação técnica do projeto

A documentação será validada pelo pipeline. Isso demonstra que CI/CD não precisa verificar apenas código: também pode garantir que artefatos mínimos de documentação continuem existindo.

9.1 README.md

O README deve ser o ponto de entrada do projeto. Ele precisa explicar o objetivo, a stack, a estrutura, como executar e quais validações existem no pipeline.

9.2 docs/INSTALLATION.md

O arquivo INSTALLATION.md deve explicar como preparar o ambiente, instalar dependências, executar backend e frontend, rodar lint, testes e build.

9.3 docs/ARCHITECTURE.md

O arquivo ARCHITECTURE.md deve explicar a arquitetura geral, separando backend, frontend, testes, documentação e pipeline CI/CD.

9.4 docs/diagrams/api-flow.md

Este arquivo deve conter um diagrama Mermaid com o fluxo geral entre usuário, frontend, backend, validações, dados em memória e resposta.

```
```mermaid
graph TD
 User[Usuário] --> Frontend[Frontend React]
 Frontend --> API[Backend Express]
 API --> Routes[Rotas da API]
 Routes --> Validation[Validações]
 Validation --> Memory[Dados em memória]
 Memory --> API
 API --> Frontend
 Frontend --> User
```
```

9.5 Prompt para preencher a documentação

Preencha a documentação do projeto FeedbackHub CI com base nos arquivos existentes.

Atualize:

- README.md
- docs/ARCHITECTURE.md
- docs/INSTALLATION.md
- docs/diagrams/api-flow.md

Considere:

- Backend com Node.js e Express
- Frontend com React e Vite
- Testes do backend e frontend
- OpenAPI em backend/openapi.yaml
- Pipeline CI/CD com GitHub Actions
- Dados armazenados em memória

Use linguagem clara, técnica e objetiva. Não invente banco de dados, autenticação ou serviços externos.

10. Arquivo .gitignore

O .gitignore evita que dependências, arquivos temporários e builds sejam enviados ao repositório.

```
node_modules/  
backend/node_modules/  
frontend/node_modules/  
frontend/dist/  
npm-debug.log  
.env  
.DS_Store
```

11. Pipeline CI/CD com GitHub Actions

O workflow será criado em .github/workflows/ci.yml. Ele será executado automaticamente em push para main e em pull_request para main.

A separação em jobs facilita a leitura do resultado. Se o backend falhar, o problema aparece no job Backend Validation. Se a documentação estiver incompleta, a falha aparece em Documentation Validation.

11.1 Estrutura dos jobs

| Job | Responsabilidade |
|--------------------------|---|
| Backend Validation | Instalar dependências, rodar lint, testes, build e validar OpenAPI. |
| Frontend Validation | Instalar dependências, rodar lint, testes e build do frontend. |
| Documentation Validation | Verificar se os documentos mínimos existem. |
| Simulated Deploy | Executar apenas se os jobs anteriores passarem. |

11.2 Arquivo .github/workflows/ci.yml

```
name: CI Pipeline
```

```
on:
```

```
  push:
```

```
    branches:
```

- main

```
  pull_request:
```

```
    branches:
```

- main


```

jobs:
  backend:
    name: Backend Validation
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: backend
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20

      - name: Install backend dependencies
        run: npm install

      - name: Run backend lint
        run: npm run lint

      - name: Run backend tests
        run: npm test

      - name: Run backend build validation
        run: npm run build

      - name: Check OpenAPI file
        run: test -f openapi.yaml

      - name: Check OpenAPI archive endpoint
        run: grep -q "/feedbacks/{id}/archive" openapi.yaml

  frontend:
    name: Frontend Validation
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: frontend
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20

      - name: Install frontend dependencies
        run: npm install

      - name: Run frontend lint
        run: npm run lint

      - name: Run frontend tests
        run: npm test

      - name: Build frontend
        run: npm run build

  documentation:
    name: Documentation Validation
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Check documentation files
        run: |
          test -f README.md
          test -d docs

```

```
test -f docs/ARCHITECTURE.md
test -f docs/INSTALLATION.md
test -f docs/diagrams/api-flow.md

deploy:
  name: Simulated Deploy
  runs-on: ubuntu-latest
  needs:
    - backend
    - frontend
    - documentation
  steps:
    - name: Simulate deploy
      run: echo "Deploy would run here after backend, frontend and documentation validation."
```

11.3 Por que usar deploy simulado?

O deploy simulado permite estudar a lógica de dependência entre jobs sem configurar servidor, cloud ou credenciais. O job final só roda quando backend, frontend e documentação passam.

12. Validação local antes do GitHub

Antes de fazer push, rode as validações localmente. Isso reduz falhas simples no GitHub Actions.

12.1 Backend

```
cd backend
npm run lint
npm test
npm run build
cd ..
```

12.2 Frontend

```
cd frontend
npm run lint
npm test
npm run build
cd ..
```

12.3 Documentação

```
test -f README.md
test -f docs/ARCHITECTURE.md
test -f docs/INSTALLATION.md
test -f docs/diagrams/api-flow.md
test -f backend/openapi.yaml
grep -q "/feedbacks/{id}/archive" backend/openapi.yaml
```

13. Enviar o projeto para o GitHub

```
git init
git add .
git commit -m "feat: create fullstack feedbackhub ci project"

git remote add origin https://github.com/SEU_USUARIO/feedbackhub-ci.git
git branch -M main
git push -u origin main
```

Substitua SEU_USUARIO pelo usuário ou organização correta do GitHub.

13.1 Ver o pipeline

No GitHub, acesse o repositório e clique na aba Actions. Abra o workflow CI Pipeline e confira os jobs:

- Backend Validation
- Frontend Validation
- Documentation Validation
- Simulated Deploy

14. Fluxo com branch e Pull Request

Depois que a main estiver funcionando, crie uma branch para simular uma alteração funcional.

```
git checkout -b feature/improve-dashboard-title
```

Altere o título em frontend/src/App.jsx:

```
<h1>Gestão de feedbacks de usuários</h1>
```

para:

```
<h1>Central de feedbacks de usuários</h1>
```

Essa alteração será usada para demonstrar como um teste de frontend pode falhar quando a interface muda, mas o teste continua esperando o texto antigo.

15. Experimentos de falha controlada

Os experimentos abaixo devem ser usados para praticar leitura de logs, identificação da causa da falha e correção. A ideia não é apenas quebrar o pipeline, mas entender qual camada foi protegida pela validação automática.

Experimento 1 - Falha no teste do frontend

Item	Descrição
O que alterar	Alterar o título principal da aplicação sem atualizar o teste.
Objetivo do experimento	Demonstrar que testes de interface protegem elementos importantes da tela.
Por que falhou	O teste em src/App.test.jsx ainda procura o texto 'Gestão de feedbacks de usuários'. Como a interface passou a exibir 'Central de feedbacks de usuários', o Vitest não encontra o elemento esperado.
Como corrigir	Atualize o teste para procurar o novo texto ou reverta o título, de acordo com a regra esperada.

Experimento 2 - Falha no lint do frontend

Item	Descrição
O que alterar	Adicionar const debugMessage = 'teste'; dentro de App.jsx sem utilizar a variável.
Objetivo do experimento	Demonstrar que o pipeline bloqueia problemas de qualidade de código antes do merge.
Por que falhou	O ESLint está configurado com no-unused-vars como erro. Variáveis declaradas e não utilizadas fazem o lint falhar.
Como corrigir	Remova a variável não utilizada ou utilize-a corretamente.

Experimento 3 - Falha no build do frontend

Item	Descrição
O que alterar	Trocar o import de FeedbackCard para '../components/CardInexistente.jsx'.
Objetivo do experimento	Demonstrar que o build detecta erros que impedem o empacotamento da aplicação.
Por que falhou	O Vite tenta resolver todos os imports. Como CardInexistente.jsx não existe, o build não consegue ser concluído.
Como corrigir	Restaure o import correto: '../components/FeedbackCard.jsx'.

Experimento 4 - Falha na documentação

Item	Descrição
O que alterar	Remover docs/ARCHITECTURE.md.
Objetivo do experimento	Demonstrar que o pipeline também pode validar artefatos mínimos de documentação.
Por que falhou	O job Documentation Validation executa test -f docs/ARCHITECTURE.md. Como o arquivo foi removido, o comando retorna erro.
Como corrigir	Restaure o arquivo docs/ARCHITECTURE.md e faça novo commit.

Experimento 5 - Falha no OpenAPI

Item	Descrição
O que alterar	Remover o trecho /feedbacks/{id}/archive do backend/openapi.yaml.
Objetivo do experimento	Demonstrar que o pipeline pode validar se uma funcionalidade importante está documentada no contrato da API.
Por que falhou	O job do backend executa grep -q "/feedbacks/{id}/archive" openapi.yaml. Se o texto não existir, o grep retorna erro e o job falha.
Como corrigir	Restaure a documentação do endpoint no backend/openapi.yaml.

16. Prompt para gerar o pipeline com IA

O prompt abaixo pode ser usado em ChatGPT, Kiro, Copilot ou Cursor para gerar o arquivo ci.yml. O importante é informar estrutura, jobs, comandos e restrições.

```
Gere o conteúdo do arquivo .github/workflows/ci.yml para uma aplicação full-stack chamada FeedbackHub CI.
```

```
Estrutura:  
- backend/  
- frontend/  
- docs/  
- README.md  
- backend/openapi.yaml
```

```
O workflow deve:  
- rodar em push para main;
```

```

- rodar em pull_request para main;
- usar Node.js 20;
- ter jobs separados para backend, frontend, documentação e deploy simulado.

Backend:
- working-directory: backend
- instalar dependências;
- rodar npm run lint;
- rodar npm test;
- rodar npm run build;
- verificar se backend/openapi.yaml existe;
- verificar com grep se backend/openapi.yaml contém /feedbacks/{id}/archive;
- se esse endpoint não existir no OpenAPI, o job backend deve falhar.

Frontend:
- working-directory: frontend
- instalar dependências;
- rodar npm run lint;
- rodar npm test;
- rodar npm run build.

Documentação:
- verificar se existem README.md, docs/ARCHITECTURE.md, docs/INSTALLATION.md e docs/diagrams/api-flow.md.

Deploy:
- criar um job final de deploy simulado com echo;
- o deploy só deve rodar se backend, frontend e documentação passarem.

Gere apenas o YAML final.

```

17. Prompt para analisar erro do GitHub Actions

Quando o pipeline falhar, copie o trecho do log e peça uma análise objetiva. O prompt deve pedir causa provável, arquivo afetado e correção recomendada.

```

Analise este erro do GitHub Actions.

Contexto:
- Projeto full-stack FeedbackHub CI
- Backend: Node.js + Express
- Frontend: React + Vite
- Pipeline com jobs de backend, frontend, documentação e deploy simulado

Log do erro:
[cole aqui o trecho do log]

Responda com:
1. Em qual job ocorreu a falha
2. Qual comando falhou
3. Qual é a causa provável
4. Qual arquivo deve ser corrigido
5. Qual comando devo rodar localmente para validar a correção

```

18. Problemas comuns e correções

Problema	Causa provável	Correção
npm ci ou npm install falha	package.json e package-lock.json inconsistentes ou dependências ausentes.	Rode npm install na pasta correta e envie package-lock.json atualizado.
Lint falha com no-unused-vars	Variável declarada e não utilizada.	Remova a variável ou use-a corretamente.
Teste do frontend falha	Texto esperado no teste não	Atualize o teste ou ajuste a interface

	corresponde ao texto renderizado.	conforme a regra esperada.
Build do frontend falha	Import inválido ou arquivo inexistente.	Corrija o caminho do import.
Documentação falha	Arquivo obrigatório removido ou no caminho errado.	Restaure README.md, docs/ARCHITECTURE.md, docs/INSTALLATION.md ou api-flow.md.
OpenAPI falha no grep	Endpoint /feedbacks/{id}/archive ausente do openapi.yaml.	Documente novamente o endpoint obrigatório.

19. Checklist final de entrega

Antes de finalizar a prática, confira os itens abaixo.

- O repositório feedbackhub-ci foi criado e está no GitHub.
- A estrutura backend/, frontend/, docs/ e .github/workflows/ existe.
- O backend possui scripts de lint, test, build e start.
- O frontend possui scripts de lint, test, build e dev.
- O arquivo backend/openapi.yaml existe e documenta /feedbacks/{id}/archive.
- README.md, docs/ARCHITECTURE.md, docs/INSTALLATION.md e docs/diagrams/api-flow.md existem.
- O arquivo .github/workflows/ci.yml possui jobs separados para backend, frontend, documentação e deploy simulado.
- As validações locais foram executadas antes do push.
- O workflow aparece na aba Actions do GitHub.
- O Pull Request mostra os checks do GitHub Actions.
- Ao menos uma falha foi analisada e corrigida com base no log.

20. Critérios de qualidade

Critério	O que será observado
Organização	Estrutura de pastas coerente com a proposta full-stack.
Backend	API executável, endpoints coerentes, testes e lint funcionando.
Frontend	Aplicação renderiza, teste mínimo passa, lint e build funcionam.
Documentação	Arquivos mínimos existem e descrevem corretamente o projeto.
Pipeline	GitHub Actions executa jobs separados e bloqueia falhas relevantes.
Análise crítica	Capacidade de interpretar logs e explicar por que uma falha ocorreu.

Resultado esperado

O objetivo da prática é compreender o funcionamento de um pipeline CI/CD completo em uma aplicação full-stack. A entrega ideal não é apenas ter um workflow passando, mas saber explicar o que cada job valida, por que uma falha ocorreu e como corrigi-la de forma técnica.